

```
//+-----+
```

```
//| Expert Advisor for Deriv Synthetic Indices |
```

```
//| Using Candlestick Pattern Recognition with Advanced Risk Management
```

```
//+-----+
```

```
#property copyright "Copyright 2023, Automated Trading Solutions"
```

```
#property link      "https://www.example.com"
```

```
#property version   "2.0"
```

```
#include <Trade\Trade.mqh>
```

```
//+-----+
```

```
//| Enumeration for pattern signals |
```

```
//+-----+
```

```
enum ENUM_PATTERN_SIGNAL
```

```
{
```

```
    PATTERN_NONE,      // No pattern detected
```

```
    PATTERN_BULLISH,    // Bullish pattern detected
```

```
    PATTERN_BEARISH,    // Bearish pattern detected
```

```
    PATTERN_CONFIRMATION // Pattern confirmation needed
```

```
};
```

```
//+-----+
```

```
//| Input Parameters |
```

```
//+-----+
```

```
input group "General Settings"
```

```
input string SymbolList = "Volatility 75 Index,Volatility 100 Index"; // Symbols to trade (comma separated)
```

```
input ENUM_TIMEFRAMES Timeframe = PERIOD_M15; // Chart timeframe
```

input int MaxTradesPerSymbol = 1; // Maximum open trades per symbol

input int TradeCooldownBars = 3; // Bars to wait between trades

input group "Pattern Settings"

input bool EnableEngulfing = true; // Enable Engulfing patterns

input bool EnableHammer = true; // Enable Hammer/Hanging Man

input bool EnableDoji = true; // Enable Doji patterns

input bool EnableStar = true; // Enable Morning/Evening Star

input bool EnableHarami = true; // Enable Harami patterns

input bool EnableTweezer = true; // Enable Tweezer patterns

input bool EnableThreeSoldiers = true; // Enable Three Soldiers/Crows

input int PatternConfirmationBars = 1; // Bars to wait for confirmation

input group "Indicator Filters"

input bool UseRSIFilter = true; // Enable RSI filter

input int RSIPeriod = 14; // RSI period

input double RSIOverbought = 70; // RSI overbought level

input double RSIOversold = 30; // RSI oversold level

input bool UseMAFilter = true; // Enable MA filter

input int MAPeriod = 50; // MA period

input ENUM_MA_METHOD MAMethod = MODE_EMA; // MA method

input group "Risk Management"

input double RiskPerTrade = 2.0; // Risk % per trade

input double FixedLotSize = 0.1; // Fixed lot size (if not using risk-based)

input bool UseRiskBasedLotSize = true; // Use risk-based position sizing

input double MaxDailyLossPercent = 5.0; // Max daily loss % before stopping

input double MaxDrawdownPercent = 20.0; // Max account drawdown % before stopping

input int MaxTradesPerDay = 10; // Maximum trades per day

input group "Exit Strategy"

input double FixedStopLoss = 50; // Fixed SL in points (0 for dynamic)

input double FixedTakeProfit = 100; // Fixed TP in points (0 for dynamic)

input bool UseATRStop = true; // Use ATR for dynamic stop

input int ATRPeriod = 14; // ATR period

input double ATRMultiplier = 1.5; // ATR multiplier for SL

input double RiskRewardRatio = 2.0; // Risk:Reward ratio for dynamic TP

input bool UseTrailingStop = true; // Enable trailing stop

input int TrailingStopDistance = 30; // Trailing stop distance (points)

input int TrailingStep = 10; // Trailing step (points)

input bool UseBreakEven = true; // Enable break-even

input int BreakEvenAt = 50; // Move to breakeven at (points)

input group "Time Restrictions"

input int TradingStartHour = 8; // Start trading hour (GMT)

input int TradingEndHour = 18; // End trading hour (GMT)

input bool UseTradingHours = true; // Enable trading hours restriction

input group "Logging & Notifications"

input bool EnableLogging = true; // Enable logging to file

input bool EnableChartDisplay = true; // Enable on-chart display

//+-----+

//| Global variables

|

```
//+-----+
```

```
string SymbolsToTrade[];
```

```
int TotalSymbols;
```

```
datetime LastTradeTime[];
```

```
int TradesToday = 0;
```

```
datetime LastTradeDay = 0;
```

```
double InitialBalance;
```

```
bool TradingAllowed = true;
```

```
int LogFileHandle = INVALID_HANDLE;
```

```
CTrade Trade;
```

```
//+-----+
```

```
//| Expert initialization function |
```

```
//+-----+
```

```
int OnInit()
```

```
{
```

```
    // Initialize logging
```

```
    if(EnableLogging)
```

```
    {
```

```
        string filename = "MT5_Bot_" + IntegerToString(GetTickCount()) + ".log";
```

```
        LogFileHandle = FileOpen(filename, FILE_WRITE|FILE_CSV|FILE_COMMON);
```

```
        if(LogFileHandle == INVALID_HANDLE)
```

```
        {
```

```
            Print("Failed to open log file: ", GetLastError());
```

```
        }
```

```
    else
```

```

{
    FileWrite(LogFileHandle, "Time", "Symbol", "Action", "Price", "LotSize", "SL", "TP", "Message");
}
}

```

// Split symbol list

```
StringSplit(SymbolList, ',', SymbolsToTrade);
```

```
TotalSymbols = ArraySize(SymbolsToTrade);
```

```
ArrayResize(LastTradeTime, TotalSymbols);
```

```
ArrayInitialize(LastTradeTime, 0);
```

// Store initial balance for drawdown calculation

```
InitialBalance = AccountInfoDouble(ACCOUNT_BALANCE);
```

// Verify symbol availability

```
for(int i = 0; i < TotalSymbols; i++)
```

```
{
```

```
    string symbol = SymbolsToTrade[i];
```

```
    if(!SymbolInfoInteger(symbol, SYMBOL_SELECT))
```

```
    {
```

```
        Print("Symbol ", symbol, " is not available in Market Watch");
```

```
        ArrayRemove(SymbolsToTrade, i, 1);
```

```
        TotalSymbols--;
```

```
        i--;
```

```
    }
```

```
}
```

```

if(TotalSymbols == 0)
{
    Alert("No valid symbols to trade. Please check Market Watch.");
    return(INIT_FAILED);
}

// Display initialization message

string msg = StringFormat("EA Initialized with %d symbols. Trading %s timeframe.",
                          TotalSymbols, EnumToString(Timeframe));

Print(msg);

return(INIT_SUCCEEDED);
}

//+-----+
//| Expert deinitialization function          |
//+-----+

void OnDeinit(const int reason)
{
    // Clean up logging

    if(LogFileHandle != INVALID_HANDLE)
    {
        FileClose(LogFileHandle);

        LogFileHandle = INVALID_HANDLE;
    }

    Print("EA Deinitialized. Reason: ", getUninitReasonText(reason));
}

```

```
}
```

```
//+-----+
```

```
//| Expert tick function |
```

```
//+-----+
```

```
void OnTick()
```

```
{
```

```
    // Check if trading is allowed (risk management)
```

```
    if(!TradingAllowed)
```

```
    {
```

```
        Comment("\nTrading Suspended by Risk Management");
```

```
        return;
```

```
    }
```

```
    // Check trading hours restriction
```

```
    if(UseTradingHours)
```

```
    {
```

```
        MqlDateTime timeStruct;
```

```
        TimeCurrent(timeStruct);
```

```
        int currentHour = timeStruct.hour;
```

```
        if(currentHour < TradingStartHour || currentHour >= TradingEndHour)
```

```
        {
```

```
            Comment("\nOutside Trading Hours");
```

```
            return;
```

```
        }
```

```
    }
```

```

// Reset daily trade counter if it's a new day

MqlDateTime today;

TimeCurrent(today);

if(today.day != LastTradeDay)

{
    TradesToday = 0;

    LastTradeDay = today.day;
}


// Check for trailing stop and breakeven opportunities

CheckTrailingStopAndBreakeven();


// Process each symbol

for(int i = 0; i < TotalSymbols; i++)

{
    string symbol = SymbolsToTrade[i];


    // Check if we can trade this symbol (cooldown, max trades)

    if(!CanTradeSymbol(symbol, i))

        continue;


    // Get current candle and previous candles for pattern detection

    MqlRates rates[];

    if(CopyRates(symbol, Timeframe, 0, 5, rates) < 5)

    {
        Print("Failed to get rates for ", symbol);
    }
}

```



```

    continue;
}

// Check for candlestick patterns
int patternSignal = CheckCandlestickPatterns(rates);

// If no pattern detected, continue
if(patternSignal == PATTERN_NONE)
    continue;

// Apply indicator filters if enabled
if(!CheckIndicatorFilters(symbol, patternSignal))
    continue;

// Calculate position size
double lotSize = CalculatePositionSize(symbol, patternSignal);
if(lotSize <= 0)
    continue;

// Calculate stop loss and take profit
double stopLoss = CalculateStopLoss(symbol, rates, patternSignal);
double takeProfit = CalculateTakeProfit(symbol, stopLoss, patternSignal);

// Execute trade
if(ExecuteTrade(symbol, patternSignal, lotSize, stopLoss, takeProfit))
{
    LastTradeTime[i] = TimeCurrent();
}

```

```

    TradesToday++;

    Print("Trade executed for ", symbol, " based on pattern signal");

}

}

// Update chart display
UpdateChartDisplay();

}

//+-----+
//| Candlestick Pattern Detection Functions |
//+-----+

int CheckCandlestickPatterns(const MqlRates &rates[])
{
    // Check for bullish patterns

    if(EnableEngulfing && IsBullishEngulfing(rates))

        return PATTERN_BULLISH;

    if(EnableHammer && IsHammer(rates[1]))

        return PATTERN_BULLISH;

    if(EnableDoji && IsDragonflyDoji(rates[1]))

        return PATTERN_BULLISH;

    if(EnableStar && IsMorningStar(rates))

        return PATTERN_BULLISH;

```

```
if(EnableHarami && IsBullishHarami(rates))
```

```
    return PATTERN_BULLISH;
```

```
if(EnableTweezer && IsTweezerBottom(rates))
```

```
    return PATTERN_BULLISH;
```

```
if(EnableThreeSoldiers && IsThreeWhiteSoldiers(rates))
```

```
    return PATTERN_BULLISH;
```

```
// Check for bearish patterns
```

```
if(EnableEngulfing && IsBearishEngulfing(rates))
```

```
    return PATTERN_BEARISH;
```

```
if(EnableHammer && IsHangingMan(rates[1]))
```

```
    return PATTERN_BEARISH;
```

```
if(EnableDoji && IsGravestoneDoji(rates[1]))
```

```
    return PATTERN_BEARISH;
```

```
if(EnableStar && IsEveningStar(rates))
```

```
    return PATTERN_BEARISH;
```

```
if(EnableHarami && IsBearishHarami(rates))
```

```
    return PATTERN_BEARISH;
```

```
if(EnableTweezer && IsTweezerTop(rates))
```

```
    return PATTERN_BEARISH;
```

```

    if(EnableThreeSoldiers && IsThreeBlackCrows(rates))

        return PATTERN_BEARISH;

    return PATTERN_NONE;
}

bool IsBullishEngulfing(const MqlRates &rates[])
{
    if(ArraySize(rates) < 2) return false;

    if(rates[1].close >= rates[1].open) return false;

    if(rates[0].close <= rates[0].open) return false;

    if(rates[0].open > rates[1].close || rates[0].close < rates[1].open)

        return false;

    return true;
}

bool IsBearishEngulfing(const MqlRates &rates[])
{
    if(ArraySize(rates) < 2) return false;

    if(rates[1].close <= rates[1].open) return false;

    if(rates[0].close >= rates[0].open) return false;

    if(rates[0].open < rates[1].close || rates[0].close > rates[1].open)

        return false;

    return true;
}

```

```
bool IsHammer(const MqlRates &candle)
{
    double bodySize = MathAbs(candle.close - candle.open);
    double lowerWick = candle.close - candle.low;
    double upperWick = candle.high - candle.open;
    if(lowerWick < 2 * bodySize) return false;
    if(upperWick > bodySize * 0.5) return false;
    return true;
}
```

```
bool IsHangingMan(const MqlRates &candle)
{
    double bodySize = MathAbs(candle.close - candle.open);
    double lowerWick = candle.open - candle.low;
    double upperWick = candle.high - candle.close;
    if(lowerWick < 2 * bodySize) return false;
    if(upperWick > bodySize * 0.5) return false;
    return true;
}
```

```
bool IsDragonflyDoji(const MqlRates &candle)
{
    if(MathAbs(candle.close - candle.open) > 0.1 * (candle.high - candle.low))
        return false;
    if(candle.high - MathMax(candle.close, candle.open) > 0.1 * (candle.high - candle.low))
        return false;
    if(MathMin(candle.close, candle.open) - candle.low < 0.6 * (candle.high - candle.low))
```

```
        return false;

    return true;

}
```

```
bool IsGravestoneDoji(const MqlRates &candle)
```

```
{
    if(MathAbs(candle.close - candle.open) > 0.1 * (candle.high - candle.low))
        return false;

    if(MathMin(candle.close, candle.open) - candle.low > 0.1 * (candle.high - candle.low))
        return false;

    if(candle.high - MathMax(candle.close, candle.open) < 0.6 * (candle.high - candle.low))
        return false;

    return true;
}
```

```
bool IsMorningStar(const MqlRates &rates[])
```

```
{
    if(ArraySize(rates) < 3) return false;

    if(rates[2].close >= rates[2].open) return false;

    double bodySize2 = MathAbs(rates[1].close - rates[1].open);

    double range2 = rates[1].high - rates[1].low;

    if(bodySize2 > 0.3 * range2) return false;

    if(rates[0].close <= rates[0].open) return false;

    if(rates[0].close < rates[2].open) return false;

    return true;
}
```

```
bool IsEveningStar(const MqlRates &rates[])
{
    if(ArraySize(rates) < 3) return false;
    if(rates[2].close <= rates[2].open) return false;
    double bodySize2 = MathAbs(rates[1].close - rates[1].open);
    double range2 = rates[1].high - rates[1].low;
    if(bodySize2 > 0.3 * range2) return false;
    if(rates[0].close >= rates[0].open) return false;
    if(rates[0].close > rates[2].open) return false;
    return true;
}
```

```
bool IsBullishHarami(const MqlRates &rates[])
{
    if(ArraySize(rates) < 2) return false;
    if(rates[1].close >= rates[1].open) return false;
    if(rates[0].close <= rates[0].open) return false;
    if(rates[0].open < rates[1].close || rates[0].close > rates[1].open)
        return false;
    return true;
}
```

```
bool IsBearishHarami(const MqlRates &rates[])
{
    if(ArraySize(rates) < 2) return false;
    if(rates[1].close <= rates[1].open) return false;
    if(rates[0].close >= rates[0].open) return false;
```

```
if(rates[0].open > rates[1].close || rates[0].close < rates[1].open)

    return false;

return true;

}
```

```
bool IsTweezerBottom(const MqlRates &rates[])

{

    if(ArraySize(rates) < 2) return false;

    if(MathAbs(rates[0].low - rates[1].low) > 0.0005) return false;

    if(rates[1].close >= rates[1].open) return false;

    if(rates[0].close <= rates[0].open) return false;

    return true;

}
```

```
bool IsTweezerTop(const MqlRates &rates[])

{

    if(ArraySize(rates) < 2) return false;

    if(MathAbs(rates[0].high - rates[1].high) > 0.0005) return false;

    if(rates[1].close <= rates[1].open) return false;

    if(rates[0].close >= rates[0].open) return false;

    return true;

}
```

```
bool IsThreeWhiteSoldiers(const MqlRates &rates[])

{

    if(ArraySize(rates) < 3) return false;

    for(int i = 0; i < 3; i++)
```



```

{
    if(rates[i].close <= rates[i].open) return false;

    if(i > 0 && rates[i].close <= rates[i-1].close) return false;
}

for(int i = 1; i < 3; i++)

{
    if(rates[i].open < rates[i-1].open || rates[i].open > rates[i-1].close)

        return false;
}

return true;
}

```

```

bool IsThreeBlackCrows(const MqlRates &rates[])

{
    if(ArraySize(rates) < 3) return false;

    for(int i = 0; i < 3; i++)

    {
        if(rates[i].close >= rates[i].open) return false;

        if(i > 0 && rates[i].close >= rates[i-1].close) return false;
    }

    for(int i = 1; i < 3; i++)

    {
        if(rates[i].open > rates[i-1].open || rates[i].open < rates[i-1].close)

            return false;
    }

    return true;
}

```

```

//+-----+

//| Trade Execution and Management Functions |

//+-----+

bool ExecuteTrade(const string symbol, const int signal, const double lotSize,
                  const double stopLoss, const double takeProfit)
{
    if(!RiskManagementCheck(symbol, lotSize))
        return false;

    double price = 0;
    double sl = 0;
    double tp = 0;

    if(signal == PATTERN_BULLISH)
    {
        price = SymbolInfoDouble(symbol, SYMBOL_ASK);
        if(stopLoss > 0) sl = price - stopLoss * _Point;
        if(takeProfit > 0) tp = price + takeProfit * _Point;

        bool result = Trade.Buy(lotSize, symbol, price, sl, tp, "Bullish Pattern");
        if(result) LogTrade(symbol, "BUY", price, lotSize, sl, tp, "Bullish pattern trade");
        return result;
    }

    else if(signal == PATTERN_BEARISH)
    {
        price = SymbolInfoDouble(symbol, SYMBOL_BID);

```

```
if(stopLoss > 0) sl = price + stopLoss * _Point;
```

```
if(takeProfit > 0) tp = price - takeProfit * _Point;
```

```
bool result = Trade.Sell(lotSize, symbol, price, sl, tp, "Bearish Pattern");
```

```
if(result) LogTrade(symbol, "SELL", price, lotSize, sl, tp, "Bearish pattern trade");
```

```
return result;
```

```
}
```

```
return false;
```

```
}
```

```
bool CheckIndicatorFilters(const string symbol, const int signal)
```

```
{
```

```
    if(UseRSIFilter)
```

```
    {
```

```
        double rsi[];
```

```
        if(CopyBuffer(iRSI(symbol, Timeframe, RSIPeriod, PRICE_CLOSE), 0, 0, 1, rsi) < 1)
```

```
            return false;
```

```
        if(signal == PATTERN_BULLISH && rsi[0] > RSIOversold)
```

```
            return false;
```

```
        if(signal == PATTERN_BEARISH && rsi[0] < RSIOverbought)
```

```
            return false;
```

```
    }
```

```
    if(UseMAFilter)
```

```

{
    double ma[];

    if(CopyBuffer(iMA(symbol, Timeframe, MAPeriod, 0, MAMethod, PRICE_CLOSE), 0, 0, 1, ma) <
1)

        return false;


    double price = SymbolInfoDouble(symbol, signal == PATTERN_BULLISH ? SYMBOL_ASK :
SYMBOL_BID);


    if(signal == PATTERN_BULLISH && price < ma[0])

        return false;


    if(signal == PATTERN_BEARISH && price > ma[0])

        return false;
}


return true;
}

```

```

double CalculatePositionSize(const string symbol, const int signal)

```

```

{
    if(!UseRiskBasedLotSize)

        return FixedLotSize;


    double price = SymbolInfoDouble(symbol, signal == PATTERN_BULLISH ? SYMBOL_ASK :
SYMBOL_BID);


    MqlRates rates[];

```

```

if(CopyRates(symbol, Timeframe, 0, 1, rates) < 1)

    return FixedLotSize;

double stopLoss = CalculateStopLoss(symbol, rates, signal);

if(stopLoss <= 0)

    return FixedLotSize;

double riskAmount = AccountInfoDouble(ACCOUNT_BALANCE) * RiskPerTrade / 100;

double tickValue = SymbolInfoDouble(symbol, SYMBOL_TRADE_TICK_VALUE);

double pointValue = SymbolInfoDouble(symbol, SYMBOL_POINT);

if(tickValue == 0 || pointValue == 0)

    return FixedLotSize;

double lotSize = riskAmount / (stopLoss * pointValue * tickValue);

double minLot = SymbolInfoDouble(symbol, SYMBOL_VOLUME_MIN);

double maxLot = SymbolInfoDouble(symbol, SYMBOL_VOLUME_MAX);

double lotStep = SymbolInfoDouble(symbol, SYMBOL_VOLUME_STEP);

lotSize = MathRound(lotSize / lotStep) * lotStep;

lotSize = MathMax(minLot, MathMin(maxLot, lotSize));

return lotSize;

}

```

```

double CalculateStopLoss(const string symbol, const MqlRates &rates[], const int signal)
{
    if(FixedStopLoss > 0)
        return FixedStopLoss;

    if(UseATRStop)
    {
        double atr[];

        if(CopyBuffer(iATR(symbol, Timeframe, ATRPeriod), 0, 0, 1, atr) < 1)
            return FixedStopLoss > 0 ? FixedStopLoss : 50;

        return atr[0] * ATRMultiplier / _Point;
    }

    if(ArraySize(rates) > 0)
    {
        if(signal == PATTERN_BULLISH)
            return (rates[0].open - rates[0].low) / _Point;
        else
            return (rates[0].high - rates[0].open) / _Point;
    }

    return 50;
}

```

```

double CalculateTakeProfit(const string symbol, const double stopLoss, const int signal)
{

```

```

if(FixedTakeProfit > 0)

    return FixedTakeProfit;

if(stopLoss <= 0)

    return 0;

return stopLoss * RiskRewardRatio;
}

//+-----+
//| Risk Management Functions |
//+-----+

bool RiskManagementCheck(const string symbol, const double lotSize)
{
    if(!TradingAllowed)

        return false;

    double dailyProfit = AccountInfoDouble(ACCOUNT_PROFIT);

    if(dailyProfit < 0 && MathAbs(dailyProfit) >= AccountInfoDouble(ACCOUNT_BALANCE) *
MaxDailyLossPercent / 100)
    {
        string msg = StringFormat("Daily loss limit reached (%.2f %s). Trading suspended.",
                                dailyProfit, AccountInfoString(ACCOUNT_CURRENCY));

        Print(msg);

        TradingAllowed = false;

        return false;
    }
}

```

```

double equity = AccountInfoDouble(ACCOUNT_EQUITY);

double balance = AccountInfoDouble(ACCOUNT_BALANCE);

double drawdownPercent = (balance - equity) / balance * 100;


if(drawdownPercent >= MaxDrawdownPercent)

{
    string msg = StringFormat("Max drawdown reached (%.2f%%). Trading suspended.",
drawdownPercent);

    Print(msg);

    TradingAllowed = false;

    return false;
}


// Calculate margin required for the potential trade

double marginRequired = 0;

if(lotSize > 0)

{
    double contractSize = SymbolInfoDouble(symbol, SYMBOL_TRADE_CONTRACT_SIZE);

    long leverage = AccountInfoInteger(ACCOUNT_LEVERAGE);

    double price = SymbolInfoDouble(symbol, SYMBOL_BID);

    if(contractSize > 0 && leverage > 0 && price > 0)

    {
        marginRequired = (contractSize * lotSize * price) / leverage;

    }

}

double marginFree = AccountInfoDouble(ACCOUNT_MARGIN_FREE);


if(marginRequired > 0 && marginFree < marginRequired * 2)

```



```

{
    Print("Not enough free margin to open new trades");
    return false;
}

return true;
}

void CheckTrailingStopAndBreakeven()
{
    for(int i = PositionsTotal()-1; i >= 0; i--)
    {
        if(PositionGetTicket(i))
        {
            string symbol = PositionGetString(POSITION_SYMBOL);
            ulong ticket = PositionGetInteger(POSITION_TICKET);
            double openPrice = PositionGetDouble(POSITION_PRICE_OPEN);
            double currentPrice = PositionGetDouble(POSITION_PRICE_CURRENT);
            double sl = PositionGetDouble(POSITION_SL);
            double tp = PositionGetDouble(POSITION_TP);
            double pointsProfit = 0;

            if(PositionGetInteger(POSITION_TYPE) == POSITION_TYPE_BUY)
            {
                pointsProfit = (currentPrice - openPrice) / _Point;

                if(UseBreakEven && sl < openPrice && pointsProfit >= BreakEvenAt)

```

```
{  
    Trade.PositionModify(ticket, openPrice, tp);  
    Print("Moved to breakeven for position #", ticket);  
}
```

```
if(UseTrailingStop && pointsProfit > TrailingStopDistance)
```

```
{  
    double newSl = currentPrice - TrailingStopDistance * _Point;  
    if(newSl > sl)  
    {  
        Trade.PositionModify(ticket, newSl, tp);  
        Print("Trailing stop updated for position #", ticket);  
    }  
}
```

```
}  
else if(PositionGetInteger(POSITION_TYPE) == POSITION_TYPE_SELL)
```

```
{  
    pointsProfit = (openPrice - currentPrice) / _Point;
```

```
    if(UseBreakEven && sl > openPrice && pointsProfit >= BreakEvenAt)
```

```
{  
    Trade.PositionModify(ticket, openPrice, tp);  
    Print("Moved to breakeven for position #", ticket);  
}
```

```
if(UseTrailingStop && pointsProfit > TrailingStopDistance)
```

```
{
```

```

double newSl = currentPrice + TrailingStopDistance * _Point;

if(newSl < sl)

{
    Trade.PositionModify(ticket, newSl, tp);

    Print("Trailing stop updated for position #", ticket);

}

}

}

}

}

}

```

```

//+-----+

```

```

//| Utility Functions |

```

```

//+-----+

```

```

bool CanTradeSymbol(const string symbol, const int symbolIndex)

```

```

{
    if(PositionsTotalForSymbol(symbol) >= MaxTradesPerSymbol)

        return false;

```

```

    if(TimeCurrent() - LastTradeTime[symbolIndex] < TradeCooldownBars *
PeriodSeconds(Timeframe))

```

```

        return false;

```

```

    if(TradesToday >= MaxTradesPerDay)

```

```

        return false;

```

```

    return true;

```

```
}
```

```
int PositionsTotalForSymbol(const string symbol)
```

```
{
```

```
    int count = 0;
```

```
    for(int i = PositionsTotal()-1; i >= 0; i--)
```

```
    {
```

```
        if(PositionGetSymbol(i) == symbol)
```

```
            count++;
```

```
    }
```

```
    return count;
```

```
}
```

```
void UpdateChartDisplay()
```

```
{
```

```
    if(!EnableChartDisplay) return;
```

```
    string comment = "\n=== Automated Trading Bot ===";
```

```
    comment += StringFormat("\nAccount Balance: %.2f", AccountInfoDouble(ACCOUNT_BALANCE));
```

```
    comment += StringFormat("\nEquity: %.2f", AccountInfoDouble(ACCOUNT_EQUITY));
```

```
    comment += StringFormat("\nTrades Today: %d/%d", TradesToday, MaxTradesPerDay);
```

```
    int totalPositions = PositionsTotal();
```

```
    if(totalPositions > 0)
```

```
    {
```

```
        comment += "\n\n=== Open Positions ===";
```

```
        for(int i = 0; i < totalPositions; i++)
```

```

{
    if(PositionGetTicket(i))
    {
        string symbol = PositionGetString(POSITION_SYMBOL);
        double profit = PositionGetDouble(POSITION_PROFIT);
        comment += StringFormat("\n%s: %.2f %s", symbol, profit,
AccountInfoString(ACCOUNT_CURRENCY));
    }
}
}

```

```

    Comment(comment);
}

```

```

void LogTrade(const string symbol, const string action, const double price,
    const double lotSize, const double sl, const double tp, const string message)
{
    if(EnableLogging && LogFileHandle != INVALID_HANDLE)
    {
        FileWrite(LogFileHandle, TimeToString(TimeCurrent()), symbol, action,
            price, lotSize, sl, tp, message);
    }
}

```

```

if(EnableChartDisplay)
{
    PrintFormat("%s - %s %s %.5f @ %.5f (SL: %.5f, TP: %.5f) - %s",
        TimeToString(TimeCurrent()), symbol, action,
        lotSize, price, sl, tp, message);
}

```

```
}
```

```
}
```

```
string getUninitReasonText(int reasonCode)
```

```
{
```

```
    string reason="";
```

```
    switch(reasonCode)
```

```
    {
```

```
        case REASON_ACCOUNT:
```

```
            reason="Account was changed"; break;
```

```
        case REASON_CHARTCHANGE:
```

```
            reason="Symbol or timeframe was changed"; break;
```

```
        case REASON_CHARTCLOSE:
```

```
            reason="Chart was closed"; break;
```

```
        case REASON_PARAMETERS:
```

```
            reason="Input parameters were changed"; break;
```

```
        case REASON_RECOMPILE:
```

```
            reason="Program "+__FILE__+" was recompiled"; break;
```

```
        case REASON_REMOVE:
```

```
            reason="Program "+__FILE__+" was removed from chart"; break;
```

```
        case REASON_TEMPLATE:
```

```
            reason="New template was applied to chart"; break;
```

```
        default:
```

```
            reason="Another reason";
```

```
    }
```

```
    return reason;
```

```
}
```